# Focused Action Refinement for Frozen 1-bit Vision-Language-Action Models

Miaomiao Dai, Yijie Liao, Weiyi Wang, Peilin Meng
Department of Electrical Engineering and Computer Science
University of Michigan
{dmmsjtu, lyjie, wweiyi, pailen}@umich.edu

Abstract—Vision-Language-Action (VLA) models enable generalist robot control but remain costly to deploy. BitVLA reduces this cost via low-bit quantization, at the expense of action precision, especially in long-horizon manipulation tasks. We propose Focused Action Refinement (FAR), a lightweight module that refines top-layer action features of a frozen 1-bit VLA backbone, improving action prediction without modifying backbone parameters. FAR combines (i) a triple-branch attention design that separates action self-attention, action–query cross-attention, and action–vision cross-attention into independent softmax branches, (ii) a patch-level top-$k$ visual selection paired with channel gating, and (iii) a dual-level residual anchoring strategy that bounds refinement for stable execution. On LIBERO-Long, FAR improves success rate from 85.8% to 88.6%, with larger gains on multi-object tasks. The module adds only 120M parameters on top of a 3.5B frozen backbone (3.4%), offering an efficient way to recover precision loss from extreme quantization.

## I. Introduction

Vision-Language-Action (VLA) models unify visual perception, language conditioning, and robot control into a single policy, enabling a wide range of manipulation tasks under natural language instructions. Recent systems such as OpenVLA and $\pi_0$ demonstrate that a single model can generalize across diverse manipulation tasks. However, their practical deployment remains constrained by model scale, as high-capacity VLAs require substantial memory and computational resources [1]–[3].

BitVLA addresses this challenge through aggressive quantization, reducing the backbone to 1-bit weights with 8-bit activations while maintaining competitive performance. Despite this efficiency gain, quantization introduces systematic degradation in action precision, particularly in long-horizon manipulation tasks involving multiple objects and spatial reasoning [4].

This raises a key question: can we recover the lost precision of a quantized VLA without retraining or modifying the backbone?

In this work, we propose Focused Action Refinement (FAR), a lightweight sidecar module that improves action prediction of a frozen 1-bit VLA policy. Instead of relearning the full control policy, FAR operates on top-layer action representations and selectively refines them using multimodal interactions with visual features. The refinement is explicitly constrained to a bounded residual around the baseline action, so the closed-loop behavior cannot drift far from the frozen policy even when the refinement output is noisy.

The central idea is to make refinement structured and selective. Drawing on FocusVLA's modality-cascaded design [5], FAR disentangles action self-attention, action–query cross-attention, and action–vision cross-attention into three independent softmax branches, preventing the query-side shortcut that can arise when modalities share a single attention. In addition, it employs patch-level top-$k$ selection and channel gating to attend to task-relevant visual evidence, and a dual-level residual anchoring mechanism to bound refinement at both the feature and action levels.

This work has two parts, aligned with the course's reproduction + extension deliverable structure:

- Reproduction. We first reproduce the LIBERO-Long rollout result of the base paper (BitVLA [4]) under the authors' official evaluation protocol, verifying the published baseline and establishing a trustworthy reference point for our extension.
- Algorithmic extension. We then propose Focused Action Refinement (FAR) as a new component on top of the reproduced baseline, and evaluate it under the same protocol to quantify its improvement.

Our contributions are summarized as follows:

- We propose a frozen-backbone action refinement module for quantized VLA policies, improving action precision without modifying backbone parameters.
- We introduce a triple-branch attention mechanism that enforces structured interactions between action, query, and visual tokens, mitigating shortcut learning.
- We design a visual refinement strategy combining patch-level top-$k$ selection with channel gating, together with dual-level residual anchoring to ensure stable closed-loop execution.
- We demonstrate improvements on LIBERO-Long, increasing success rate from 85.8% to 88.6%, with the largest gains on multi-object manipulation tasks [6].

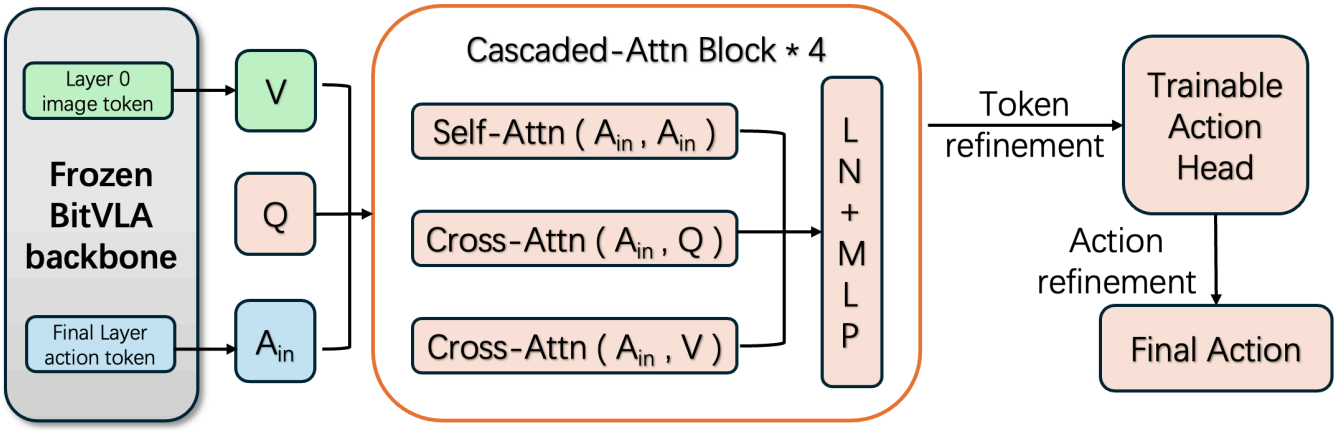


Fig. 1. Overview of the FAR framework. A frozen VLA backbone produces intermediate tokens and baseline actions. A lightweight refinement module processes these tokens through a cascaded attention encoder and produces a corrected action, which is bounded around the baseline via residual anchoring.

## II. Related Work

### A. Vision-Language-Action Models

Vision-Language-Action (VLA) models connect language-conditioned perception with continuous robot control, enabling generalist manipulation policies under natural language instructions. OpenVLA demonstrates that large pretrained multimodal architectures can be adapted into unified robot policies, achieving strong generalization across tasks [2]. BitVLA extends this line of work toward efficient deployment by applying aggressive low-bit quantization, significantly reducing memory and computation while maintaining competitive performance on LIBERO benchmarks [4]. In parallel, models such as $\pi_0$ explore alternative policy representations, including flow-matching-based generation for manipulation tasks [1]. Despite these advances, a key challenge remains: how to preserve action accuracy under strict efficiency constraints, especially in long-horizon and multi-object manipulation settings.

### B. Efficient Adaptation of Large Models

Parameter-efficient adaptation methods, such as LoRA, reduce the number of trainable parameters during fine-tuning by introducing low-rank updates into pretrained models [7]. Similar approaches have been widely applied in vision-language and robotics models to enable efficient adaptation without full retraining. However, these methods still modify the backbone representations, which may alter the deployment characteristics of highly optimized models such as quantized VLAs. In contrast, our approach preserves a strictly frozen backbone and shifts all adaptation into a separate refinement module. This design is particularly relevant for quantized policies, where maintaining the original inference cost and numerical behavior is critical.

### C. Structured Attention for Multimodal Policies

Structured attention mechanisms in multimodal models have been shown to mitigate shortcut learning and improve cross-modal grounding. The Multimodal Transformer introduced cross-modal attention for unaligned multimodal sequences [8], and subsequent work has studied how to control information flow between modalities in VLA policies. Most directly related to our work, FocusVLA [5] proposed a modality-cascaded attention with patch-level focus and channel gating for end-to-end VLA training, showing that explicit modality separation and selective visual utilization substantially improve action prediction.

Our method adapts FocusVLA's attention design to a frozen-backbone inference setting. Rather than retraining a full VLA policy, we introduce a lightweight refinement module on top of a quantized BitVLA backbone. The module inherits the cascaded attention and top-$k$ plus channel-gate visual selection from FocusVLA, but operates purely as a post-hoc corrector that never modifies the backbone's 1-bit weights. In this sense FAR sits closer to a bounded residual corrector than to a standalone policy.

## III. Algorithmic Extension: FAR

### A. Problem Setup

We consider a frozen quantized Vision-Language-Action (VLA) policy

$$f_{\text{base}} : (I, w, p, l) \mapsto a_{\text{base}}, \tag{1}$$

where $I$ denotes RGB observations, $w$ wrist-view images, $p$ proprioception, and $l$ the language instruction. The model predicts chunked actions $a_{\text{base}} \in \mathbb{R}^{K \times A}$ with horizon $K = 8$ and action dimension $A = 7$. In addition to the final action, the frozen backbone exposes intermediate representations:

- early visual tokens $H_0 \in \mathbb{R}^{B \times N_v \times D}$,

- top-layer action tokens $H_{\text{top}} \in \mathbb{R}^{B\times K\times A\times D}$,

where $D$ is the hidden dimension and $N_v$ is the number of visual tokens. Our goal is to learn a refinement function

$$a_{\text{final}} = g(H_0, H_{\text{top}}, a_{\text{base}}) \tag{2}$$

that improves action accuracy while keeping the backbone fully frozen.

### B. FAR Framework

The proposed Focused Action Refinement (FAR) module operates as a lightweight sidecar on top of the frozen backbone (Fig. 1). It consists of three stages: (1) feature extraction, (2) token refinement via cascaded attention, and (3) bounded action correction. We first project the extracted features into a refinement space of dimension $R$:

$$A = W_A H_{\text{top}} + E_{\text{chunk}} + E_{\text{dim}}, \tag{3}$$
$$Q = Q_{\text{emb}} + E_{\text{chunk}}, \tag{4}$$
$$V = W_V H_0, \tag{5}$$

where $A \in \mathbb{R}^{B\times 56\times R}$ are flattened action tokens, $Q \in \mathbb{R}^{B\times K\times R}$ are learned chunk query tokens, and $V \in \mathbb{R}^{B\times N_v\times R}$ are visual tokens.

### C. Cascaded Attention Refinement

A key design in FAR is to decouple multimodal interactions into separate attention branches. Instead of mixing all tokens in a single attention, we explicitly model:

$$H_A = \text{SelfAttn}(A_{\text{in}}, A_{\text{in}}), \tag{6}$$
$$H_Q = \text{CrossAttn}(A_{\text{in}}, Q), \tag{7}$$
$$H_V = \text{CrossAttn}(A_{\text{in}}, V). \tag{8}$$

Each branch uses an independent softmax, preventing competition between modalities and ensuring that visual information is explicitly incorporated. The outputs are fused as:

$$H_{\text{fuse}} = \text{MLP}([\text{LN}(H_A), \text{LN}(H_Q), \text{LN}(H_V)]), \tag{9}$$
$$A_{\text{out}} = A_{\text{in}} + H_{\text{fuse}} + \text{FFN}(A_{\text{in}} + H_{\text{fuse}}). \tag{10}$$

To improve spatial selectivity, we restrict attention to the top-$k$ visual tokens:

$$\tilde{S}_{ij} = \begin{cases} S_{ij}, & j \in \text{TopK}(S_i), \\ -\infty, & \text{otherwise}, \end{cases} \tag{11}$$

followed by

$$H_V^{\text{raw}} = \text{softmax}(\tilde{S})V. \tag{12}$$

We further apply channel-wise gating:

$$H_V = H_V^{\text{raw}} \odot \sigma(\text{MLP}(A_{\text{in}})), \tag{13}$$

which enables fine-grained filtering of visual features.

TABLE I
Main FAR hyperparameters.

| Parameter | Value |
|---|---|
| Backbone hidden dim $D$ | 2560 |
| Refinement dim $R$ | 640 |
| Heads | 8 |
| Layers | 4 |
| Chunk size | 8 |
| Action dimension | 7 |
| Top-$k$ patches | 256 |
| Token scale $\delta$ | 0.30 |
| Pose scale $\rho$ | 0.10 |

### D. Action Decoding and Anchoring

The refined latent is reshaped as

$$z_{\text{action}} \in \mathbb{R}^{B\times K\times A\times R}. \tag{14}$$

We apply token-level refinement:

$$\Delta h = \delta \cdot \tanh(\text{MLP}([z_{\text{action}}, H_{\text{top}}])), \tag{15}$$

and obtain

$$H_{\text{refined}} = H_{\text{top}} + \Delta h. \tag{16}$$

The final action is predicted as

$$a_{\text{refine}} = \text{Head}(H_{\text{refined}}), \tag{17}$$

and anchored to the baseline:

$$a_{\text{final}} = a_{\text{base}} + \rho \cdot \tanh(a_{\text{refine}} - a_{\text{base}}). \tag{18}$$

This dual-level residual design ensures that refinement remains stable and bounded.

## IV. Experimental Setup

### A. Dataset and Benchmark

We evaluate our method on the LIBERO-Long benchmark, using the libero_10_no_noops dataset for training and the libero_10 suite for rollout evaluation. The benchmark consists of 10 long-horizon manipulation tasks, each evaluated with 50 trials, resulting in 500 episodes in total.

The policy takes as input $224 \times 224$ RGB observations from both agent-view and wrist-view cameras, along with proprioceptive state and a language instruction. It predicts an 8-step chunked action sequence with 7 DoF, executed in an open-loop manner.

### B. Model Configuration

We use a frozen BitVLA backbone pretrained on LIBERO-Long. The backbone is a 30-layer model with approximately 3.5B parameters under low-bit quantization.

The FAR module introduces approximately 120M trainable parameters (3.4% of the backbone size). Key hyperparameters are summarized in Table I.

### C. Training Details

We adopt a strict frozen-backbone setting:

$$(H_0, H_{\text{top}}, a_{\text{base}}) = f_{\text{base}}(x), \tag{19}$$

where extracted features are detached from gradient flow.

The final action is modeled as a bounded residual:

$$a_{\text{final}} = a_{\text{base}} + \rho \cdot \tanh(a_{\text{refine}} - a_{\text{base}}). \tag{20}$$

The training objective combines pose regression and gripper classification:

$$\mathcal{L}_{\text{pose}} = \text{SmoothL1}(a_{\text{pose}}, a_{\text{gt}}), \tag{21}$$

$$\mathcal{L}_{\text{grip}} = \text{BCE}(a_{\text{grip}}, a_{\text{gt}}), \tag{22}$$

$$\mathcal{L} = \mathcal{L}_{\text{pose}} + \lambda \mathcal{L}_{\text{grip}}, \tag{23}$$

with $\lambda = 1.0$.

We optimize the model using AdamW with separate learning rates for the encoder ($2 \times 10^{-4}$) and action head ($5 \times 10^{-5}$), and apply cosine decay with linear warmup (500 steps over 10k total steps).

Training uses batch size 8 with gradient accumulation (effective batch size 16). Gradient clipping ($\tau = 1.0$) and residual anchoring are applied to ensure stability. The full 10,000-step training completes in approximately 3 hours on a single NVIDIA RTX 5090 (32 GB). Closed-loop rollout evaluation on all 500 LIBERO-Long episodes takes approximately 2.7 hours on the same hardware.

## V. Results

### A. Paper Reproduction

As the first deliverable, we reproduce the LIBERO-Long rollout result of the base paper (BitVLA [4]) using the authors' official evaluation script at repos/BitVLA/.../run_libero_eval_bitnet.py. We run 500 rollout episodes (10 tasks × 50 trials, seed=7) on the official ft-libero-long-bf16 checkpoint with center_crop=True, num_open_loop_steps=8, and num_steps_wait=10 — matching the BitVLA paper's evaluation protocol exactly. Our reproduction obtains **85.8**% (429/500) success, closely matching the BitVLA paper's reported "without robotics pretraining" result of 87.6% on the same suite. The small gap is consistent with simulator non-determinism (MuJoCo / driver version) across independent runs. This reproduced baseline serves as the reference point against which our extension is measured.

### B. Extension Results

Table II compares the reproduced baseline against our proposed extension, FAR, under the same 500-episode protocol.

FAR improves the overall success rate by **+2.8** percentage points (14 additional successful episodes) on top of the reproduced baseline. This demonstrates that a lightweight refinement module can recover part of the precision loss introduced by aggressive quantization without modifying the frozen backbone.

TABLE II
Overall success rate: reproduced baseline vs. our extension.

| Method | Success | Total | Success Rate |
|---|---|---|---|
| BitVLA baseline (reproduced) | 429 | 500 | 85.8% |
| FAR (our extension) | 443 | 500 | 88.6% |

TABLE III
Per-task performance on LIBERO-Long.

| Task | Baseline | FAR | Δ |
|---|---|---|---|
| Task 0 | 84% | 92% | +8 |
| Task 1 | 96% | 100% | +4 |
| Task 2 | 96% | 92% | -4 |
| Task 3 | 94% | 86% | -8 |
| Task 4 | 76% | 92% | +16 |
| Task 5 | 100% | 98% | -2 |
| Task 6 | 68% | 80% | +12 |
| Task 7 | 82% | 88% | +6 |
| Task 8 | 72% | 74% | +2 |
| Task 9 | 90% | 84% | -6 |

### C. Per-Task Analysis

Table III reports per-task performance across all tasks.

FAR improves performance on the majority of tasks, with the largest gains on Task 4 (+16) and Task 6 (+12), which correspond to more challenging scenarios involving multi-object interactions and spatial reasoning. This suggests that the refinement module is particularly effective when precise visual grounding is required.

Performance decreases are observed on a few tasks (e.g., Task 3 and Task 9), indicating that refinement is not uniformly beneficial and may occasionally introduce over-correction when the baseline policy is already near-optimal.

Overall, FAR improves performance primarily on harder tasks while maintaining competitive performance on easier ones, supporting the hypothesis that refinement is most effective when quantization errors accumulate in long-horizon or spatially complex scenarios.

## VI. Discussion

FAR improves performance primarily on tasks involving multi-object interactions and spatial reasoning. In particular, Tasks 4 and 6 show the largest gains (+16 and +12), suggesting that the proposed visual focus mechanism effectively recovers spatial precision lost due to quantization. By selectively attending to relevant regions, the model is better able to ground actions in complex scenes.

At the same time, performance degrades on a few tasks (e.g., Task 3 and Task 9), which involve tighter temporal dependencies. In these cases, refinement may introduce slight over-correction, interfering with stable execution when the baseline policy is already well-optimized.

The patch-level top-$k$ selection plays a key role in reducing noise from irrelevant regions, improving robustness in cluttered environments. However, aggressive filtering may also remove useful contextual information, which can contribute to performance drops in certain scenarios.

Residual anchoring ensures that the refined action remains close to the baseline:

$$|a_{\text{final}} - a_{\text{base}}| \leq \rho \tag{24}$$

This constraint prevents large deviations and maintains rollout stability. Empirically, FAR improves performance without introducing catastrophic failures.

Finally, FAR remains computationally efficient, introducing only 120M trainable parameters (3.4% of the backbone) and requiring approximately 3 hours of training on a single RTX 5090. This highlights that meaningful performance gains can be achieved through lightweight refinement rather than full-model retraining.

### A. Limitations and Future Directions

FAR operates under a strictly frozen backbone, which fundamentally bounds its achievable performance by the information already encoded in $H_0$ and $H_{\text{top}}$. If aggressive 1-bit quantization has destroyed information that is simply not present in these features, no post-hoc refinement can recover it. A complementary direction is end-to-end joint training of the backbone and refinement module with action supervision, which could in principle recover more of the performance gap to full-precision VLAs, at the cost of losing the drop-in deployability and 1-bit memory footprint that motivated this work.

Our evaluation uses a single random seed (seed=7) and a single LIBERO suite (LIBERO-Long). Multi-seed variance is not measured, and generalization to LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and real-robot execution remains untested. We also observe a persistent offline–online gap: pose MAE on held-out demonstrations does not reliably predict closed-loop rollout success, so checkpoint selection must be done via costly rollout evaluation rather than cheap offline metrics. Finally, FAR assumes access to the backbone's intermediate activations $H_0$ and $H_{\text{top}}$; closed-source or API-only VLAs that expose only the final action would require additional feature-probing effort before FAR can be applied.

## VII. Conclusion

We present Focused Action Refinement (FAR), a lightweight module for improving frozen 1-bit Vision-Language-Action models. By combining structured attention, visual focus, and bounded residual refinement, FAR recovers part of the precision loss introduced by aggressive quantization.

Experiments on LIBERO-Long demonstrate consistent improvements, particularly on tasks requiring spatial reasoning and multi-object interaction, while maintaining low computational overhead.

Future work includes (i) extending FAR to other frozen VLA backbones such as OpenVLA and $\pi_0$ under various quantization schemes, (ii) exploring joint backbone–refinement training to push absolute performance further while preserving deployment efficiency, and (iii) developing time-adaptive refinement schedules (e.g., a chunk-step-dependent $\rho$) to reduce over-correction on pose-sensitive closure tasks.

## References


[1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter et al., "Pi0: A vision-language-action flow model for general robot control," arXiv preprint, 2024.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi et al., "OpenVLA: An open-source vision-language-action model," arXiv preprint, 2024.

[3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," in Conference on Robot Learning (CoRL), 2023, pp. 2165–2183.

[4] H. Wang, C. Xiong, R. Wang, and X. Chen, "BitVLA: 1-bit vision-language-action models for robotic manipulation," arXiv preprint, 2025.

[5] Y. Zhang, W. Yuan, Y. Zhang, X. Zhang, and J. Wan, "FocusVLA: Focused visual utilization for vision-language-action models," arXiv preprint, 2026.

[6] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 44 776–44 791.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in International Conference on Learning Representations (ICLR), 2022.

[8] Y.-H. H. Tsai, S. Bai, P. P. Liang, J. Z. Kolter, L.-P. Morency, and R. Salakhutdinov, "Multimodal transformer for unaligned multimodal language sequences," in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), 2019, pp. 6558–6569.